

ATS RESUME PERCENTAGE MATCHER

Overview

The system extracts resume and job description data from a database, analyzes them using an ATS (Applicant Tracking System) function, calculates match percentages, and updates the results back into the database.

It takes data from the database which contains two tables:

1. HRIS_VACANCY (VACANCY_ID, POSITION, AVAILABILITY, DESCRIPTION, STATUS, CREATED_DT, MODIFIED_DT, CREATED_BY, MODIFIED_BY, VACANCY_STATUS, EVALUATE_FLAG, PERCENTAGE_MATCH)
2. HRIS_VACANCY_APPLICATION (ID, POSITION, FULL_NAME, GENDER, EMAIL, CONTACT_NUMBER, COVER_LETTER, RESUME_PATH, STATUS, APPLICANT_QUESTION, REFERENCE_FROM)

View Table is created from these two tables “RESUME_DETAILS” (ID, APPLICANT, POSITION, JOBDESCRIPTION, RESUME, FLAG, PERCENTAGE_MATCH)

Example of resume_details:

ID	APPLICANT	POSITION	JOBDESCRIPTION	RESUME	FLAG	PERCENTAGE_MATCH
1	Prasanna Ghimire	Machine learning Engineer	We are looking 2 year+ experience in ML	<pg_resume_path.pdf>	N	85
2	Diwas Adhikari	Flutter Developer	We are looking an expert in mobile app development	<da_resume_path.pdf>	N	95

Libraries Used:

- os: Provides functions for interacting with the operating system.
- google.generativeai: Library for interacting with Google's Generative AI models.
- pandas (pd): Used for data manipulation and analysis.
- json: Provides functions for parsing and handling JSON data.
- time: Provides time-related functions such as sleep.
- pdfplumber: allows for easy extraction of text and data from PDFs
- cx_Oracle: Used for python application to interact with Oracle database
- dotenv: Used to load environment variables where credentials are kept.

To install these dependencies you may use: *< pip install -r requirements.txt >*

1. ats_func_4.py

This python file consists of various functions to matching percentage. It consists function to pa

At first genai configuration. You may need GOOGLE_API_KEY. Create a new GOOGLE_API_KEY and store it into .env file and load it using load_dotenv().

Functions:

1. def safe_json_parse(response_text)

Purpose:

Safely parses a JSON string while handling formatting inconsistencies.

Parameters:

- response_text (*str*): The text containing JSON.

Returns:

- A dictionary if parsing succeeds, else {}.

How it Works:

- Direct Parsing: Attempts `json.loads(response_text)`.
- Markdown Extraction: Finds JSON inside triple backticks (````json ... ````).
- Braces Extraction: Searches for `{...}` and tries parsing.

2. def extract_structured_data(text)

Purpose:

Extracts and validates structured resume data from text using a generative AI model, ensuring relevant experience and skills are correctly categorized.

Approach:

- Prompting AI: Generates a structured JSON response containing skills, education, and experience while filtering out irrelevant experience (e.g., freelancing, internships).
- Skill Extraction: Includes explicit skills and inferred ones from job experience and projects.
- Validation & Parsing: Uses `safe_json_parse()` to ensure valid JSON.
- Retries: Attempts extraction up to 3 times with exponential backoff in case of errors.

3. `def parse_date (date_str):`

Purpose:

Parses a date string into a datetime object, handling multiple formats and edge cases.

Approach:

- Handles Empty Strings: Returns the current date if input is empty.
- Supports "Present": If "Present" is given, returns the current date.
- Multiple Date Formats: Attempts parsing with common formats:
 1. MM/YYYY → "01/2023" → January 2023
 2. YYYY/MM → "2023/01" → January 2023
 3. Month YYYY (short and full) → "Jan 2023", "January 2023" → January 2023
 4. YYYY (year-only) → "2023" → January 1, 2023
- Error Handling: If parsing fails, returns the current date as a fallback.

4. `def calculate_experiences (experience)`

Purpose:

From the total experience that is relevant captured from resume, it checks for the overlapping time period and calculate total experience accordingly to avoid double counting.

Approach:

- **Filter Relevant Experiences:**
 1. Ignores experiences marked as "relevant": false.
 2. Extracts start and end dates using `parse_date()`.
- **Sort & Merge Overlapping Date Ranges:**
 1. Sorts experiences by start date.
 2. Merges overlapping periods to prevent duplication.
- **Compute Experience Duration:**
 1. Calculates the total months for non-overlapping periods.
 2. Converts total months into years (months / 12).

5. `def get_percentage_match (JobDesc, Resume_Data)`

Purpose:

Calculates how well a resume matches a job description by analyzing skills, education, experience, and keyword relevance.

Approach:

1. Generates a Prompt for LLM (Gemini-1.5-Flash):

- Compares structured resume data (resume_data) with job description (job_desc).
- Evaluates skills, education, experience, and career stability.
- Assigns a match percentage (0-100).
- Provides a stability score and detailed breakdown.
- Generates matching based on prompt and comparing resume data and job description

2. Retries on API Failures (Up to 5 Attempts):

- Implements exponential backoff for error handling.

3. Parses and Validates Response JSON:

- Extracts and bounds values between 0-100.
- Returns structured data with strengths, weaknesses, and a detailed analysis.

6. `def analyze_resume (file_path, job_description)`

Description:

This function analyzes a resume (PDF) and compares it to a job description to assess how well the resume matches the job. It calculates total experience, match scores, and provides strengths, weaknesses, and detailed analysis.

Parameters:

- file_path (str): Path to the resume (PDF).
- job_description (str): Text of the job description.

Extract PDF:

Utilized pdfplumber library to extract contents of PDF into text;

Function call:

1. `extract_structured_data(text)`
2. `calculate_experience(resume_data.get("experience", []))`

Returns:

A dictionary with:

- Overall match percentage
- Total experience
- Relevant experience count
- Breakdown of scores
- Skills, education, strengths, weaknesses
- Detailed analysis
- Score Breakdown

2. resume_bulk_analysis_4.py

- This file consists of function to connect Oracle database with required credentials, update_database, process resumes.

i) `def get_db_connection():`

Purpose:

Get the oracle database connection from the .env file from os.getenv(). Get DB_USER, DB_PASS, HOST, PORT, SERVICE. It uses cx_Oracle library to connect it into database.

Returns:

An active database connection

ii) `def update_database(id_value, overall_match):`

Purpose:

The main purpose of this function is to update the database after percentage matching is calculated. It fires the update query into database where percentage match with respect to id will get stored over database.

Query Used:

```
UPDATE RESUME_DETAILS  
SET PERCENTAGE_MATCH = :1, FLAG = 'Y'  
WHERE ID = :2 AND FLAG = 'N'
```

It executes above query using cursor.execute() and commits in database.

iii) `def process_resume(input_file, output_file):`

Purpose:

The purpose is to read the file, analyze the resume, get the required data in specific format, update database and outputs excel file.

The process_resumes function reads an Excel file containing resumes and job descriptions, analyzes each resume against the corresponding job description, and saves the results to an output file. It also updates the database with the match percentage and handles errors gracefully.

Introduce Delay: Implements an exponential backoff delay ($2^{\text{index}} + \text{random.uniform}(0,1)$) before processing to limit the threshold for llm.

Parameters

- **input_file (str):** Path to the input Excel file containing resumes and job descriptions.
- **output_file (str):** Path to save the processed results as an Excel file.

Function call:

This function calls following functions:

- analyze_resume
- update_database

Workflow

1. Reads the input Excel file into a DataFrame.
2. Iterates through each resume entry and introduces a delay before processing.
3. Analyzes the resume against the job description to extract key insights such as:
 - Match percentage
 - Stability score
 - Total experience
 - Number of companies worked at
 - Strengths and weaknesses
4. Stores the results and updates the database with the match percentage.
5. Handles errors by logging failed entries with an "Error" status.
6. Saves the results to the specified output file.

Output

- An Excel file containing the processed results with details such as:
 - Match percentage
 - Stability score
 - Total experience
 - Strengths and weaknesses
 - Score breakdown
 - Detailed analysis

Main Function:

- At first connection to DB is done. And the data we want to fetch from Oracle database, Write query accordingly.

```
SELECT * FROM RESUME_DETAILS WHERE FLAG='N'
```

- execute and read sql query using pandas read_sql(). Save it to excel file and work on it.

How It Works

Step 1: Extract Resume Data

- The `analyze_resume()` function extracts structured data (skills, education, and experience) from each resume. This is done using the `extract_structured_data()` function, which formats the resume text into a structured JSON format.

Step 2: Match Resume to Job Description

- After extracting structured data, the resume is analyzed against the job description using the `get_match_percentage()` function. This function calculates the match percentage based on skills, experience, and education, as well as additional factors like career stability.

Step 3: Update Database

- Once the resume is analyzed, the `update_database()` function updates the database with the calculated match percentage.

Step 4: Batch Processing

- The `process_resumes()` function handles batch processing of multiple resumes, invoking `analyze_resume()` for each resume file and updating the database with the results.

Environment Variables

- **GOOGLE_API_KEY**: The API key for Google's Generative AI service.
- **DB_USER**: Database username.
- **DB_PASS**: Database password.
- **HOST**: Database host.
- **PORT**: Database port.
- **SERVICE**: Database service name.

Make sure to set these environment variables in your `.env` file before running the code.

Conclusion

This system automates the process of analyzing resumes and matching them against job descriptions. It provides valuable insights into the match percentage, strengths, weaknesses, and overall stability of candidates. The results are stored in a database for further review and use